

CSA0506 - DATABASE MANAGEMENT SYSTEMS LAB

Experiment No: 15

DCL Commands - GRANT, REVOKE

Aim:

To learn how to use various DCL commands such as GRANT and REVOKE.

Procedure:

Data Control Language (DCL) is used to control privileges in the database. To perform any operation in the database, such as creating tables, sequences or views, a user needs privileges. Privileges are of two types:

- System: Includes permissions for creating a session, table, etc., and all other types of system privileges.
- Object: Includes permissions for any command or query to perform an operation on the database tables.

In DCL we have two commands:

- GRANT: Used to provide a user access privileges or other privileges for the database.
- REVOKE: Used to take back permissions from a user.

1. Allow a user to create a session:

```
GRANT CREATE SESSION TO username;
```

2. Allow a user to create a table:

```
GRANT CREATE TABLE TO username;
```

3. Provide the user with space on tablespace to store the table (allowing a user to create a table is not enough to start storing data - the user must also be given privileges to use the available tablespace):

```
ALTER USER username QUOTA UNLIMITED ON SYSTEM;
```

(Generally, unlimited quota is provided to Admin users.)

4. Grant all privileges to a user (SYSDBA is a set of privileges that has all the permissions in it):

```
GRANT sysdba TO username;
```

5. Grant permission to create any table (bypassing reserved system-table name restrictions):

GRANT CREATE ANY TABLE TO username;

6. Grant permission to drop any table:

GRANT DROP ANY TABLE TO username;

7. To take back permissions from a user, use the REVOKE command:

REVOKE CREATE TABLE FROM username;

Output:

```
mysql>
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW GRANTS FOR 'student1'@'localhost';
+-----+
| Grants for student1@localhost |
+-----+
| GRANT USAGE ON *.* TO 'student1'@'localhost' |
| GRANT ALL PRIVILEGES ON 'labdb'.* TO 'student1'@'localhost' |
+-----+
2 rows in set (0.01 sec)

mysql> REVOKE ALL PRIVILEGES
      -> ON labdb.*
      -> FROM 'student1'@'localhost';
Query OK, 0 rows affected (0.02 sec)

mysql> SHOW GRANTS FOR 'student1'@'localhost';
+-----+
| Grants for student1@localhost |
+-----+
| GRANT USAGE ON *.* TO 'student1'@'localhost' |
+-----+
1 row in set (0.00 sec)
```

Result:

Thus the DCL commands GRANT and REVOKE were executed successfully.

Experiment No: 17

High Level Programming Extensions - Functions

Aim:

To implement Functions using a program in MySQL.

Procedure:

A function is a subprogram that computes and returns a value. The CREATE FUNCTION statement is used in MySQL to support UDFs (User-Defined Functions). A UDF can be regarded as an external stored function.

MySQL stored function syntax:

```
CREATE FUNCTION function_name(param1, param2, ...)
```

```
RETURNS datatype [NOT] DETERMINISTIC
```

```
BEGIN
```

```
... statements ...
```

```
END;
```

Problem 1: Create a function that returns the level of a customer based on the credit limit (use the IF statement to determine the credit limit):

- If credit limit > 50000 then customer_level = PLATINUM
- If credit limit >= 10000 AND credit limit <= 50000 then customer_level = GOLD
- If credit limit < 10000 then customer_level = SILVER

Recursion in MySQL Procedures (MySQL version should be >= 5): the recursion count limit must be set using the system parameters:

```
SET @@GLOBAL.max_sp_recursion_depth = 255;
```

```
SET @@session.max_sp_recursion_depth = 255;
```

Problem 2: Write a recursive MySQL procedure to compute the factorial of a number.

Output:

```
mysql>
mysql> DELIMITER ;
mysql> SELECT CustomerLevel(70000);
+-----+
| CustomerLevel(70000) |
+-----+
| PLATINUM              |
+-----+
1 row in set (0.00 sec)

mysql> SELECT CustomerLevel(25000);
+-----+
| CustomerLevel(25000) |
+-----+
| GOLD                 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT CustomerLevel(5000);
+-----+
| CustomerLevel(5000)  |
+-----+
| SILVER                |
+-----+
1 row in set (0.00 sec)
```

```
mysql>
mysql> DELIMITER ;
mysql> CALL factorial(5);
+-----+
| Factorial |
+-----+
|         120 |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Result:

Thus the Functions using a program in MySQL were executed successfully.

Experiment No: 18

High Level Language Extension with Cursors

Aim:

To implement Cursors using a program in MySQL.

Procedure:

A Cursor is a temporary memory or temporary work station. It is allocated by the database server at the time of performing DML (Data Manipulation Language) operations on a table by the user. Cursors are used to store database tables.

Implicit Cursors: Also known as default cursors of SQL Server. These cursors are allocated by SQL Server when the user performs DML operations.

Explicit Cursors: Created by users whenever required, and are used for fetching data from a table in a row-by-row manner. Steps to create an explicit cursor:

- Declare Cursor Object - Syntax: DECLARE cursor_name CURSOR FOR SELECT * FROM table_name
- Open Cursor Connection - Syntax: OPEN cursor_connection
- Fetch data from the cursor using FIRST, LAST, NEXT, PRIOR, ABSOLUTE n or RELATIVE n
- Syntax: FETCH NEXT/FIRST/LAST/PRIOR/ABSOLUTE n/RELATIVE n FROM cursor_name
- Close Cursor Connection - Syntax: CLOSE cursor_name
- Deallocate Cursor Memory - Syntax: DEALLOCATE cursor_name

Problem 1: Write a Cursor program using MySQL to retrieve the email-ids (build an email list) of employees from the employees table.

Solution:

```
CREATE TABLE employees(id INTEGER, Name VARCHAR(100), email VARCHAR(100));
```

```
INSERT INTO employees(id, Name, email) VALUES(1, 'Harry Potter', 'pharry@warnerbros.com');
```

```
INSERT INTO employees(id, Name, email) VALUES(2, 'Clark Kent', 'kclark@dccomics.com');
```

```
INSERT INTO employees(id, Name, email) VALUES(3, 'Tony Stark', 'stony@marvel.com');
```

```

DELIMITER $$

CREATE PROCEDURE build_email_list (INOUT email_list VARCHAR(4000))
BEGIN
    DECLARE v_finished INTEGER DEFAULT 0;
    DECLARE v_email VARCHAR(100) DEFAULT "";

    -- declare cursor for employee email
    DECLARE email_cursor CURSOR FOR
        SELECT email FROM employees;

    -- declare NOT FOUND handler
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_finished = 1;

    OPEN email_cursor;
    get_email: LOOP
        FETCH email_cursor INTO v_email;
        IF v_finished = 1 THEN
            LEAVE get_email;
        END IF;
        -- build email list
        SET email_list = CONCAT(v_email, ',', email_list);
    END LOOP get_email;
    CLOSE email_cursor;
END$$

DELIMITER ;

-- Calling the procedure and getting the email list
SET @email_list = "";
CALL build_email_list(@email_list);
SELECT @email_list;

```

Output:

```
mysql>
mysql> SELECT @email_list;
+-----+
| @email_list |
+-----+
| pharry@warnerbros.com;kclark@dccomics.com;stony@marvel.com; |
+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM employees;
+----+-----+-----+
| id  | name      | email |
+----+-----+-----+
| 1   | Harry Potter | pharry@warnerbros.com |
| 2   | Clark Kent  | kclark@dccomics.com  |
| 3   | Tony Stark  | stony@marvel.com     |
+----+-----+-----+
3 rows in set (0.00 sec)
```

Result:

Thus the Cursor program using MySQL was executed successfully.

Experiment No: 19

Trigger

Aim:

To implement a Trigger in MySQL.

Procedure:

A trigger or database trigger is a stored program executed automatically to respond to a specific event, e.g., insert, update or delete, occurring in a table.

Syntax - Create Trigger:

```
CREATE TRIGGER trigger_name trigger_time trigger_event ON table_name
FOR EACH ROW
BEGIN
    ...
END;
```

Program 1: Create a trigger in MySQL to log the changes of the STUDENT table with fields ID, Name and Email. Also create a new table named EMPLOYEES_AUDIT to keep the changes of the employee table. Create a BEFORE UPDATE trigger that is invoked before a change is made to the employees table.

Output:

```
mysql>
mysql> SELECT @email_list;
+-----+
| @email_list |
+-----+
| pharry@warnerbros.com;kclark@dccomics.com;stony@marvel.com; |
+-----+
1 row in set (0.00 sec)
```



```
mysql>
mysql> DELIMITER ;
mysql> UPDATE student
    -> SET email='arun123@gmail.com'
    -> WHERE id=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM student_audit;
+-----+-----+-----+-----+
| id    | name  | email          | action_time          |
+-----+-----+-----+-----+
| 1     | Arun  | arun@gmail.com | 2026-08-04 10:52:01 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Result:

Thus the trigger was executed successfully.

Experiment No: 20

MySQL String Functions - REPLACE, REPEAT, REVERSE, RIGHT, LEFT, RPAD, LPAD

Aim:

To implement the Replace, Repeat, Reverse, Right, Left, Rpad and Lpad string functions using MySQL.

Procedure:

REPLACE()

Replaces all occurrences of a substring within a string.

Syntax: REPLACE(str, from_str, to_str)

REPEAT()

Repeats a string for a specified number of times. Returns NULL if either argument is NULL.

Syntax: REPEAT(str, count)

REVERSE()

Returns a given string with the order of the characters reversed.

Syntax: REVERSE(str)

RIGHT()

Extracts a specified number of characters from the right side of a given string.

Syntax: RIGHT(str, len)

LEFT()

Returns a specified number of characters from the left of a given string.

Syntax: LEFT(str, len)

RPAD()

Pads a string from the right with padstr up to length len.

Syntax: RPAD(str, len, padstr)

LPAD()

Left-pads a string with padstr up to length len.

Syntax: LPAD(str, len, padstr)

ASCII()

Returns the numeric value of the leftmost character of the string str. Returns 0 for an empty string, NULL if str is NULL.

Syntax: ASCII(str)

BIN()

Returns a string representation of the binary value of N (a BIGINT). Returns NULL if N is NULL.

Syntax: BIN(N)

OCT()

Returns a string representation of the octal value of N (a BIGINT). Returns NULL if N is NULL.

Syntax: OCT(N)

Output:

```
mysql> SELECT REPLACE('DATABASE', 'DATA', 'FILE');
+-----+
| REPLACE('DATABASE', 'DATA', 'FILE') |
+-----+
| FILEBASE                             |
+-----+
1 row in set (0.00 sec)

mysql> SELECT REPEAT('SQL', 3);
+-----+
| REPEAT('SQL', 3) |
+-----+
| SQLSQLSQL        |
+-----+
1 row in set (0.00 sec)

mysql> SELECT REVERSE('MYSQL');
+-----+
| REVERSE('MYSQL') |
+-----+
| LQSYM            |
+-----+
1 row in set (0.00 sec)

mysql> SELECT RIGHT('DATABASE', 4);
+-----+
| RIGHT('DATABASE', 4) |
+-----+
| BASE                 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT LEFT('DATABASE', 4);
+-----+
| LEFT('DATABASE', 4) |
+-----+
| DATA               |
+-----+
1 row in set (0.01 sec)
```

```

1 row in set (0.01 sec)

mysql> SELECT RPAD('DBMS',10,'*');
+-----+
| RPAD('DBMS',10,'*') |
+-----+
| DBMS*****          |
+-----+
1 row in set (0.00 sec)

mysql> SELECT LPAD('DBMS',10,'*');
+-----+
| LPAD('DBMS',10,'*') |
+-----+
| *****DBMS         |
+-----+
1 row in set (0.01 sec)

mysql> SELECT ASCII('A');
+-----+
| ASCII('A') |
+-----+
|          65 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT BIN(10);
+-----+
| BIN(10) |
+-----+
| 1010    |
+-----+
1 row in set (0.01 sec)

mysql> SELECT OCT(10);
+-----+
| OCT(10) |
+-----+
| 12      |
+-----+
1 row in set (0.01 sec)

```

Result:

Thus the String Functions such as Replace, Repeat, Reverse, Right, Left, Rpad and Lpad were executed successfully.

Experiment No: 21

MySQL String Functions - SPACE, SUBSTR, UPPER, LOWER, TRIM, LENGTH

Aim:

To implement the Space, Substr, Upper, Lower, Trim and Length string functions using MySQL.

Procedure:

SPACE()

Returns a string containing the number of spaces specified in the argument.

Syntax: SPACE(N)

SUBSTRING() / SUBSTR()

Returns a specified number of characters from a particular position of a given string. SUBSTR() is a synonym for SUBSTRING().

Syntax: SUBSTRING(str, pos, len) / SUBSTR(str, pos, len)

UPPER()

Converts all characters in a string to uppercase.

Syntax: UPPER(str)

LOWER() / LCASE()

Converts all characters in a string to lowercase.

Syntax: LOWER(str) / LCASE(str)

TRIM()

Returns a string after removing all prefixes or suffixes from the given string.

Syntax: TRIM([{BOTH | LEADING | TRAILING} [remstr] FROM] str)

RTRIM()

Removes trailing spaces from a given string.

Syntax: RTRIM(str)

LTRIM()

Removes leading space characters of a string.

Syntax: LTRIM(str)

LENGTH()

Returns the length of a given string (in bytes).

Syntax: LENGTH(str)

BIT_LENGTH()

Returns the length of the string str in bits.

Syntax: BIT_LENGTH(str)

CHAR_LENGTH()

Returns the length of the string str, measured in characters (a multi-byte character counts as one character).

Syntax: CHAR_LENGTH(str)

CONCAT()

Returns the string resulting from concatenating one or more arguments.

Syntax: CONCAT(str1, str2, ...)

INSTR()

Returns an integer indicating the position of the first occurrence of a substring within a string.

Syntax: INSTR(str, substr)

LOCATE()

Returns the position of the first occurrence of a substring within a string, optionally starting from a given position.

Syntax: LOCATE(substr, str, pos)

MID()

Extracts a substring from a string, given the start position and length.

Syntax: MID(str, pos, len)

POSITION()

Returns the position of a substring within a string.

Syntax: POSITION(substr IN str)

Output:

```
mysql> USE labdb;
Database changed
mysql> USE labdb;
Database changed
mysql>
mysql> -- LENGTH()
mysql> SELECT LENGTH('DATABASE');
+-----+
| LENGTH('DATABASE') |
+-----+
| 8 |
+-----+
1 row in set (0.01 sec)

mysql>
mysql> -- LOWER()
mysql> SELECT LOWER('DATABASE');
+-----+
| LOWER('DATABASE') |
+-----+
| database |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- UPPER()
mysql> SELECT UPPER('database');
+-----+
| UPPER('database') |
+-----+
| DATABASE |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- CONCAT()
mysql> SELECT CONCAT('SAVEETHA',' UNIVERSITY');
+-----+
| CONCAT('SAVEETHA',' UNIVERSITY') |
+-----+
| SAVEETHA UNIVERSITY |
+-----+
```

```
mysql> SELECT SUBSTRING('DATABASE',1,4);
```

SUBSTRING('DATABASE',1,4)
DATA

```
1 row in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- INSTR()
```

```
mysql> SELECT INSTR('DATABASE','BASE');
```

INSTR('DATABASE','BASE')
5

```
1 row in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- TRIM()
```

```
mysql> SELECT TRIM('  MYSQL ');
```

TRIM(' MYSQL ')
MYSQL

```
1 row in set (0.00 sec)
```

```
mysql>
```

```
mysql> -- FORMAT()
```

```
mysql> SELECT FORMAT(1234567.891,2);
```

FORMAT(1234567.891,2)
1,234,567.89

```
1 row in set (0.00 sec)
```



```
mysql>
mysql> -- NOW()
mysql> SELECT NOW();
+-----+
| NOW() |
+-----+
| 2026-08-04 10:57:43 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- CURDATE()
mysql> SELECT CURDATE();
+-----+
| CURDATE() |
+-----+
| 2026-08-04 |
+-----+
1 row in set (0.01 sec)
```

Result:

Thus the String Functions such as Space, Substr, Upper, Lower, Trim and Length were executed successfully.